UNIVERSITY OF LEEDS

COMP1860: Building Our Digital World Computer Systems & Architecture

S1.W5.L1 Hardware Description Language & Computer Aided Design LAB 1

Learning Outcomes

UNIVERSITY OF LEEDS

- Analyse and design a combinational logic circuit
- Utilize a hardware simulator to design, test, and analyze the behavior of a digital circuit.
- Implement and debug a combinational logic circuits using HDL.

Text Book

UNIVERSITY OF LEEDS

Nand2Tetris HDL Guide / Documentation:

The Elements of Computing Systems /
Appendix 2: HDL

Overview

UNIVERSITY OF LEEDS

- Introduce Computer Aided Design (CAD) and Hardware Description Language (HDL)
- Introduce a hardware simulator
- Learn how to write HDL files
- Learn how to run simulations of simple logic circuits
- Start building the chips for the Hack computer

CAD

UNIVERSITY OF LEEDS

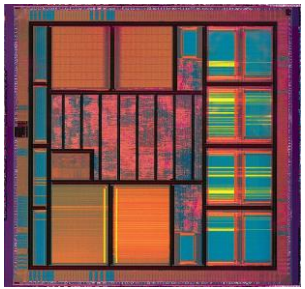
- Computer Aided Design (CAD) refers to the process of design and testing of systems using computers
- CAD may automate some of the design and testing process
- Helps to ensure that the design conforms to a specification
- Encourages reuse of common components
- Often supports simulations

Hardware Description Language

UNIVERSITY OF LEEDS

- Hardware Description Language (HDL) is a language used to specify（指定） the structure or behaviour of a digital circuit
- HDL specification can be simulated（模拟）, and the specified circuit can then be evaluated（评估）against a range of tests to ensure compliance to a specification（符合规范）
- HDL can often be synthesised onto(合成到) a Field Programmable Gate Array (FPGA) or a dedicated（专用） integrated circuit (e.g. fabricated on a silicon die)



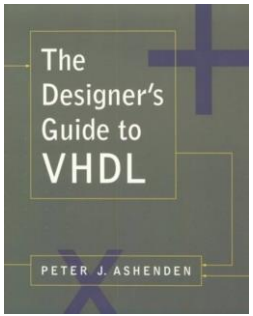


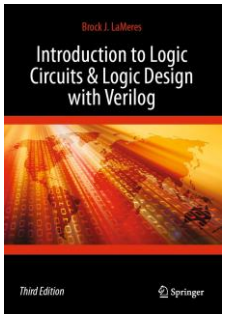
By Altera Corporation - Altera Corporation, CC BY 3.0, https://commons.wikimedia.org/w/index.php?curid=6642224
CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=1340494

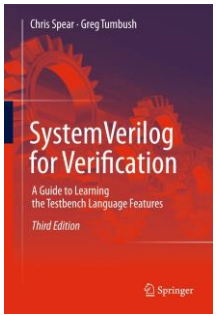
Hardware Description Language

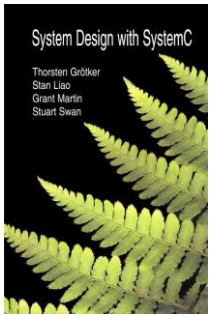
UNIVERSITY OF LEEDS

- Hardware Description Languages are widely used in industry and have evolved over the past four decades
- Commonly used HDLs include VHDL, Verilog, SystemVerilog, SystemC







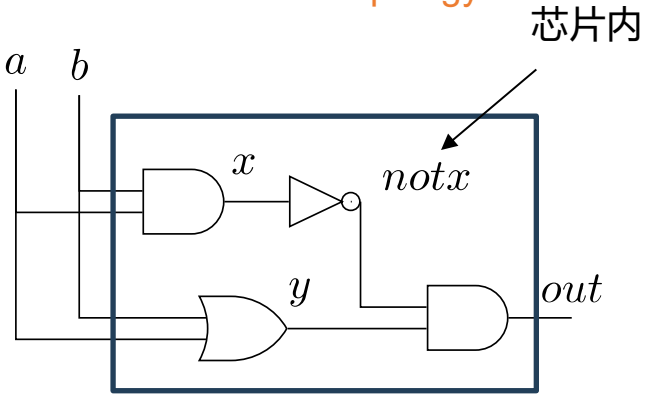


- Throughout this module, we will use a didactic language which we will refer simply as Hardware Description Language, or HDL

Hardware Description Language

UNIVERSITY OF LEEDS

- HDL models can define the internal structure of a circuit and typically consist of 2 parts
  - A *header* section - defining the interface of the circuit
  - A *parts* section - defining the names and internal topology




```
/**
 * Xor gate.
 */

CHIP Xor {
    IN a, b;
    OUT out;

    PARTS:
    And (a=a, b=b, out=x);
    Or (a=a, b=b, out=y);
    Not (in=x, out=notx);
    And (a=y, b=notx, out=out);
}
```

Header
定义芯片外引脚

Parts：定义芯片内元器件和其输入输出

Nand2Tetris HDL Guide / Documentation:

The Elements of Computing Systems / Appendix 2: HDL

HDL文件格式

UNIVERSITY OF LEEDS

```
/**
 * 注释
 */
CHIP 芯片名 {
    IN a, b;    // 芯片外输入引脚 (pin)
    OUT out;    // 芯片外输出引脚 (pin)

    PARTS:      // 定义芯片内的元器件和元器件输入
    And (a=a, b=b, out=x);    // 引入一个And gate和它的输入输出引脚
    Or (a=a, b=b, out=y);     // 引入一个Or gate和它的输入输出引脚
    Not (in=x, out=notx);     // 引入一个Not gate和它的输入输出引脚
    MyHalfAdd(a=a, b=b, s=s, c=c) // 引入自己的半加器芯片
    And (a=y, b=notx, out=out); //芯片内的最终输出要和芯片外的输出对应
}

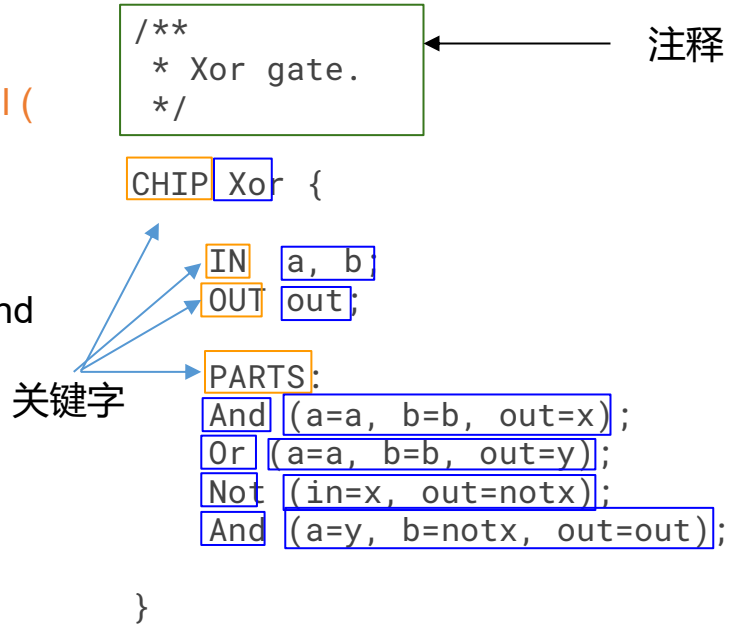
// 常见gate模拟器都已经内置，自己写的芯片HDL文件可放同目录下，用文件名引用
```

Defining a chip in HDL

UNIVERSITY OF LEEDS

- Each chip is defined in a separate text file
- A chip called Xxx is defined in a file called Xxx.hdl (大小写敏感!!)
- Keywords are capitalised（大写）
- Identifier name may be any sequence of letters and digits not starting with a digit **(no underscores)**
- Whitespace has no meaning（空格没有意义）
- Lines end with semicolons（行末用；结束）
- Comments are provided in a variety of way

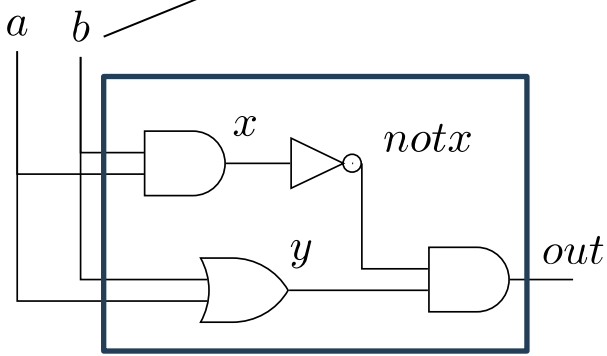
```
//  Comments to end of line
/* Comment until closing */
/** API documentation */
```



Defining a chip in HDL

UNIVERSITY OF LEEDS

- Internal chip parts（内部芯片部件）
  - `parts pin name = chips pin name`
- Internal pins（内部引脚）
  - Visible only within the chip（芯片内可见）
  - No need to define them, they are generated when required



A	B	P
0	0	0
0	1	1
1	0	1
1	1	0

DNF: (¬A∧B) ∨ (A∧¬B)

CNF: (¬A ∨¬B) ∧ (A ∨ B)

本例题是用CNF来实现逻辑电路的

```
CHIP Xor {

    IN a, b;
    OUT out;

    PARTS:
    And (a=a, b=b, out=x);
    Or (a=a, b=b, out=y);
    Not (in=x, out=notx);
    And (a=y, b=notx, out=out);

}
```

Xor.hdl

Part pin　　Internal pins

如何设计一个Chip

UNIVERSITY OF LEEDS

1. 定义chip外部的input和output引脚

```
CHIP Xor {

    IN a, b;
    OUT out;
```

2. 写出Chip的真值表、 DNF/CNF

A	B	P
0	0	0
0	1	1
1	0	1
1	1	0

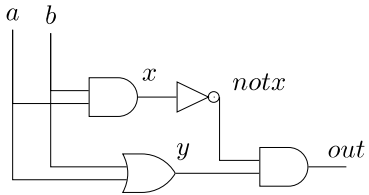
DNF: $(\neg A \wedge B) \vee (A \wedge \neg B)$

CNF: $(\neg A \vee \neg B) \wedge (A \vee B)$

本例题是用CNF来实现逻辑电路的

3. 用Kmap化简逻辑表达式（选DNF还是CNF，主要看哪个更简单）

4. 在Parts:里面实现逻辑表达式




```
PARTS:
And (a=a, b=b, out=x);
Or (a=a, b=b, out=y);
Not (in=x, out=notx);
And (a=y, b=notx, out=out);
```

5. 编写script，测试真值表

Xor.tst

```
load Xor.hdl,
output-file Xor.out,
compare-to Xor.cmp,
output-list a b out;
set a 0, set b 0, eval, output;
set a 0, set b 1, eval, output;
set a 1, set b 0, eval, output;
set a 1, set b 1, eval, output;
```

Xor.out

a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

compare

Xor.cmp

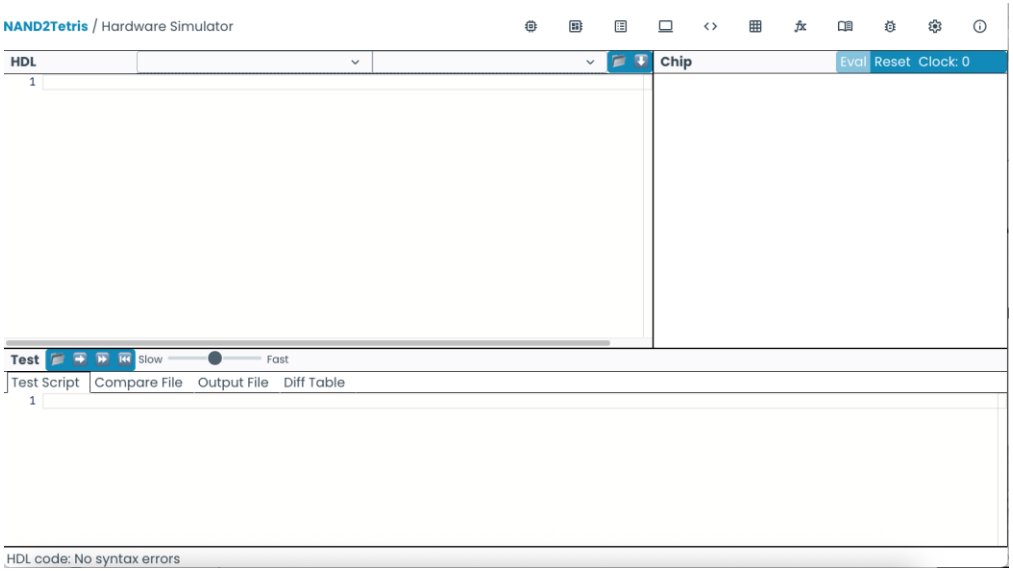
a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

Hardware Simulator

UNIVERSITY OF LEEDS

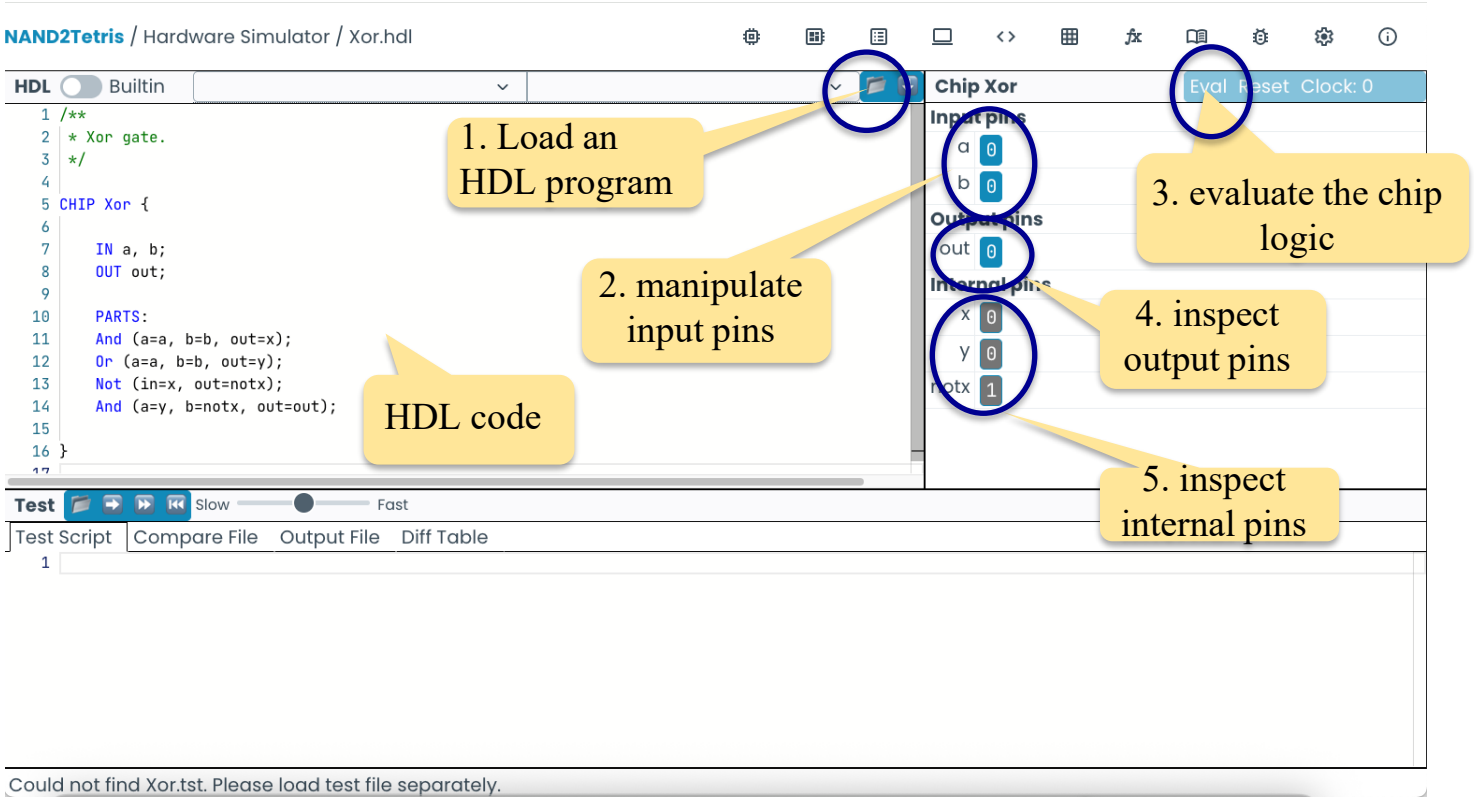
(Works in Chrome or Edge not FireFox)

In your web Browser goto https://nand2tetris.github.io/web-ide/chip/ to launch the new Integrated Development Environment for Nand2Tetris.



Interactive Simulation

UNIVERSITY OF LEEDS



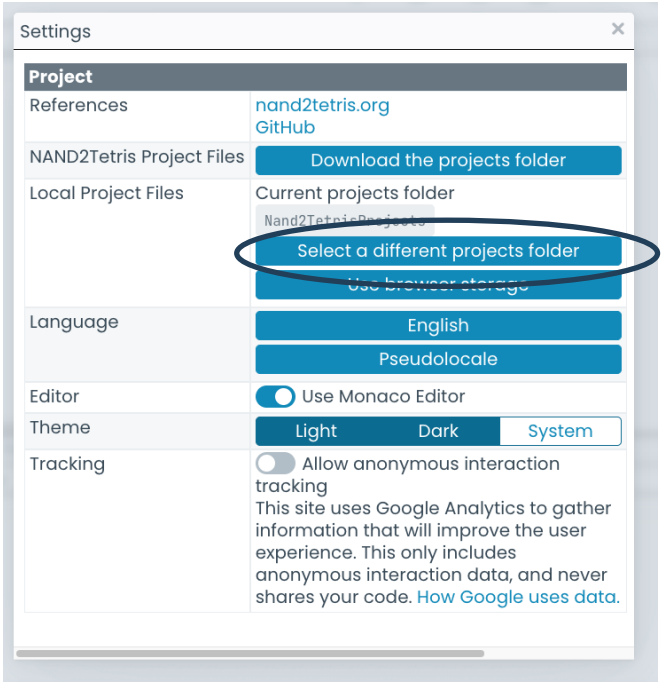
Hardware Simulator

UNIVERSITY OF LEEDS

默认情况下，此设置仅允许您处理与本书相关的项目，但我们可以更改此设置，以允许我们也处理自己的项目。

To do this

1. Click settings
2. Click Select projects folder and select or create the folder wish to store your projects in.这应该是您下载COMP1860第一学期文件的文件夹.
3. Close Settings
4. 不幸的是，您不能在模拟器中从头开始创建新文件，文件必须已经存在于您的文件夹中.
5. 要直接打开HDL文件，您现在可以选择文件夹符号



Obtaining the COMP1860 files

UNIVERSITY OF LEEDS

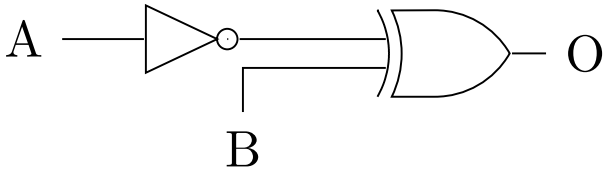
You can obtain the files you need from Minerva, under Learning Resources.

The files required for each week can be found in the folder for that week.

Your turn

UNIVERSITY OF LEEDS

Create an HDL file for the following circuit.



The chip should be called ***first*** and therefore the filename **must** be ***first.hdl*****.** We have provided an empty file for you with the correct name. **The case of the letters matters. The filename and chip name must match exactly.**

Click on the folder symbol to open the file **first.hdl** and then add the contents for your HDL in the HDL window in the simulator.

If you prefer, you can also create and edit files in a text editor such as Visual Studio Code.

Your turn

UNIVERSITY OF LEEDS

- 一旦你创建了HDL代码，我们需要测试你的芯片是否在模拟中工作。
- 将你的第一个.hdl代码加载到模拟器中，看看它是否为不同的输入值产生了预期的输出
- 您可以单击输入引脚的值，在0和1之间切换它们的值，然后单击Eval以获得输出。
- 检查芯片是否为A和B的输入值的每种可能组合产生了正确的O值。

A	B	O
0	0	1
0	1	0
1	0	0
1	1	1

Script-based Testing

UNIVERSITY OF LEEDS

Xor.hdl

```
CHIP Xor {
    IN a, b;
    OUT out;

    PARTS:
    Not (in=a, out=nota);
    Not (in=b, out=notb);
    And (a=a, b=notb, out=aAndNotb);
    And (a=nota, b=b, out=notaAndb);
    Or  (a=aAndNotb, b=notaAndb, out=out);
}
```

Xor.tst

```
load Xor.hdl,
output-file Xor.out,
output-list a b out;
set a 0, set b 0, eval, output;
set a 0, set b 1, eval, output;
set a 1, set b 0, eval, output;
set a 1, set b 1, eval, output;
```

test script

Xor.out

```
| a | b |out|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |
```

Output File, created by the test script, as a side-effect of the simulation process

Script file format

UNIVERSITY OF LEEDS

Xor.tst

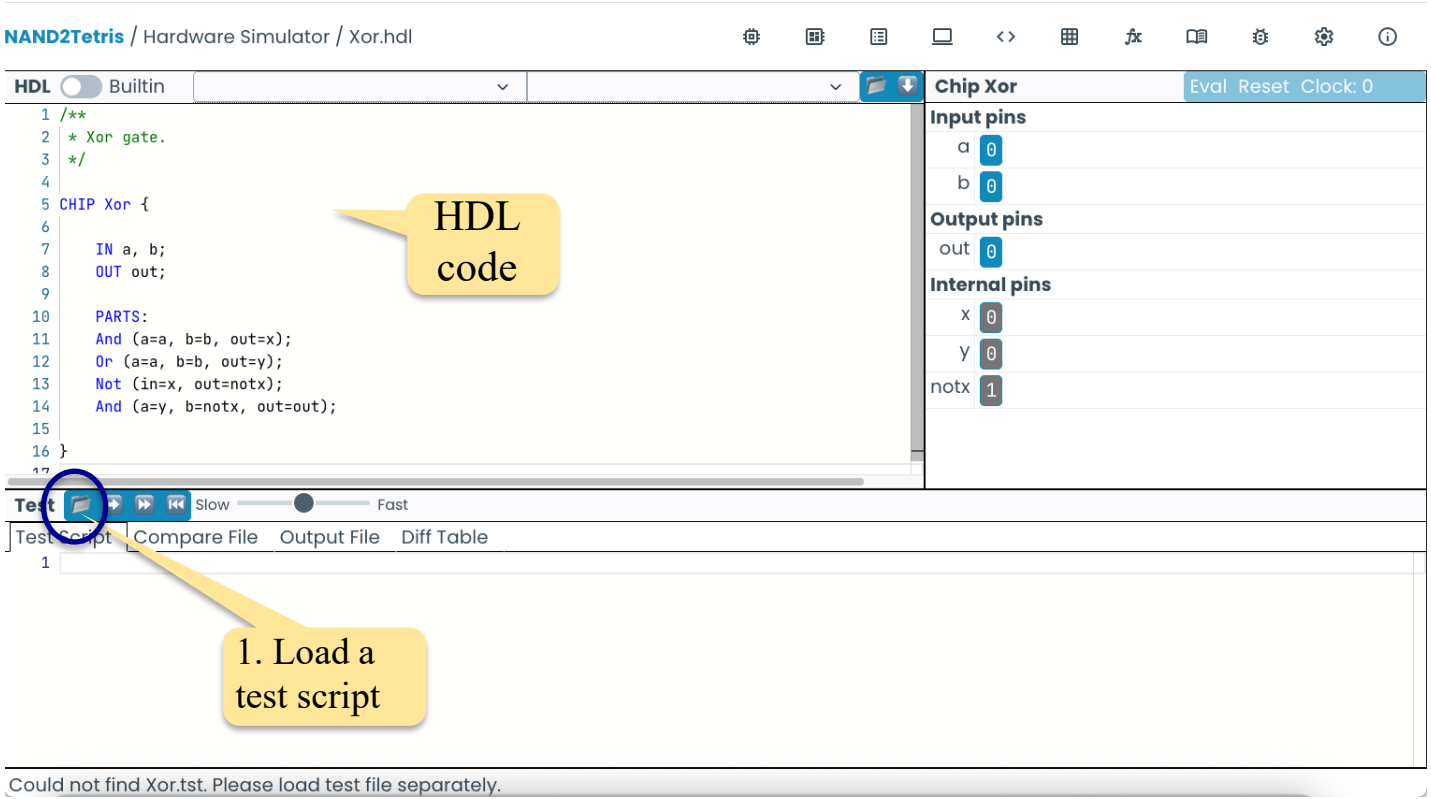
```
// 注意 load,output-file,output-list是一个完整的句子，中间用，隔开，最后加上;表示句子结束。
load Xor.hdl,                          // 加载hdl文件
output-file Xor.out,                   // 设置hdl真值表输出文件
output-list a b out;                   // 设置hdl真值表的列

// 下面是按真值表的行，逐行测试。
// set设置输入信号0/1;eval按输入执行逻辑，得到结果;output输出文件
// 注意 set,set,eval,output是一个完整的句子，中间用，隔开，最后加上;表示句子结束。
set a 0, set b 0, eval, output;
set a 0, set b 1, eval, output;
set a 1, set b 0, eval, output;
set a 1, set b 1, eval, output;
```

数是10进制数

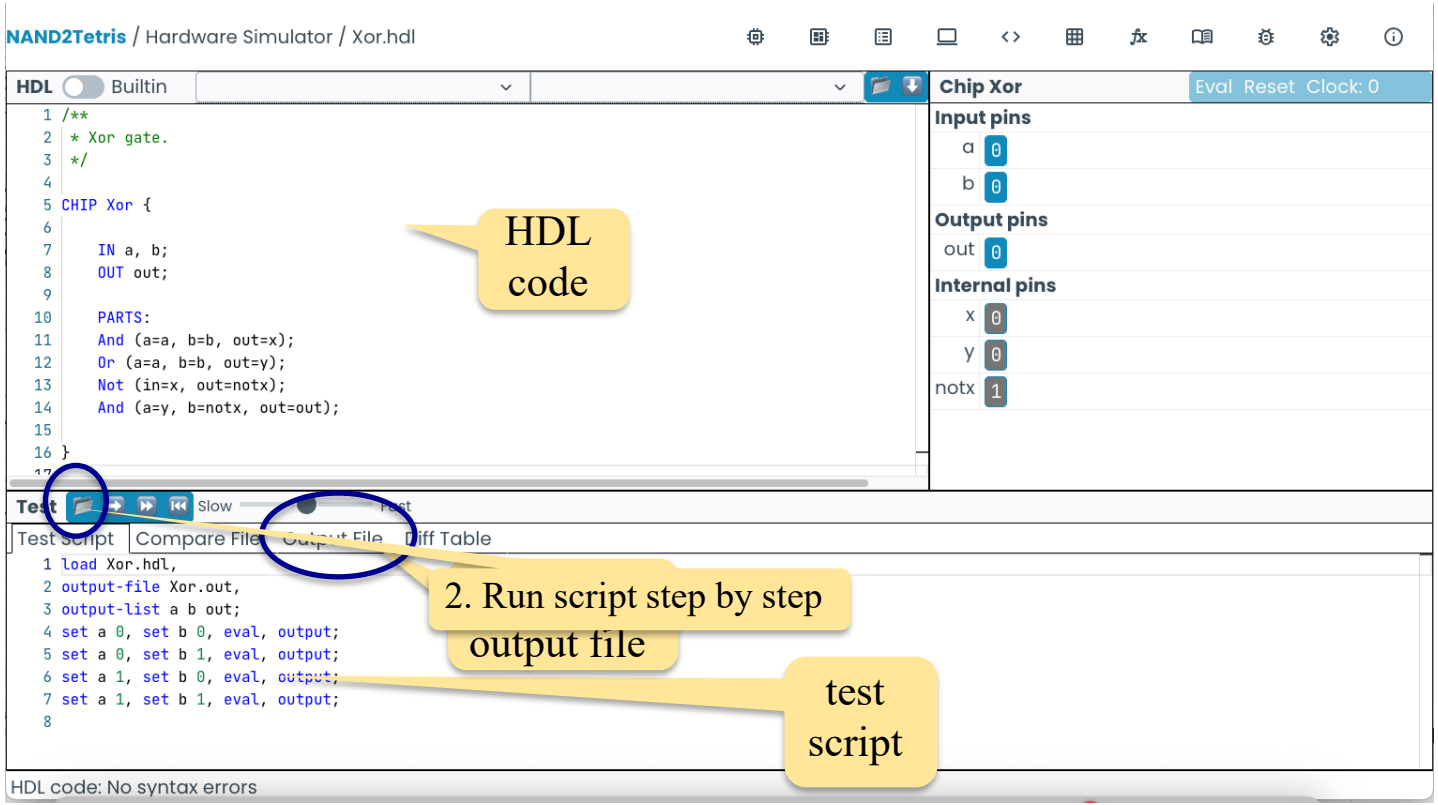
Script-based Testing

UNIVERSITY OF LEEDS



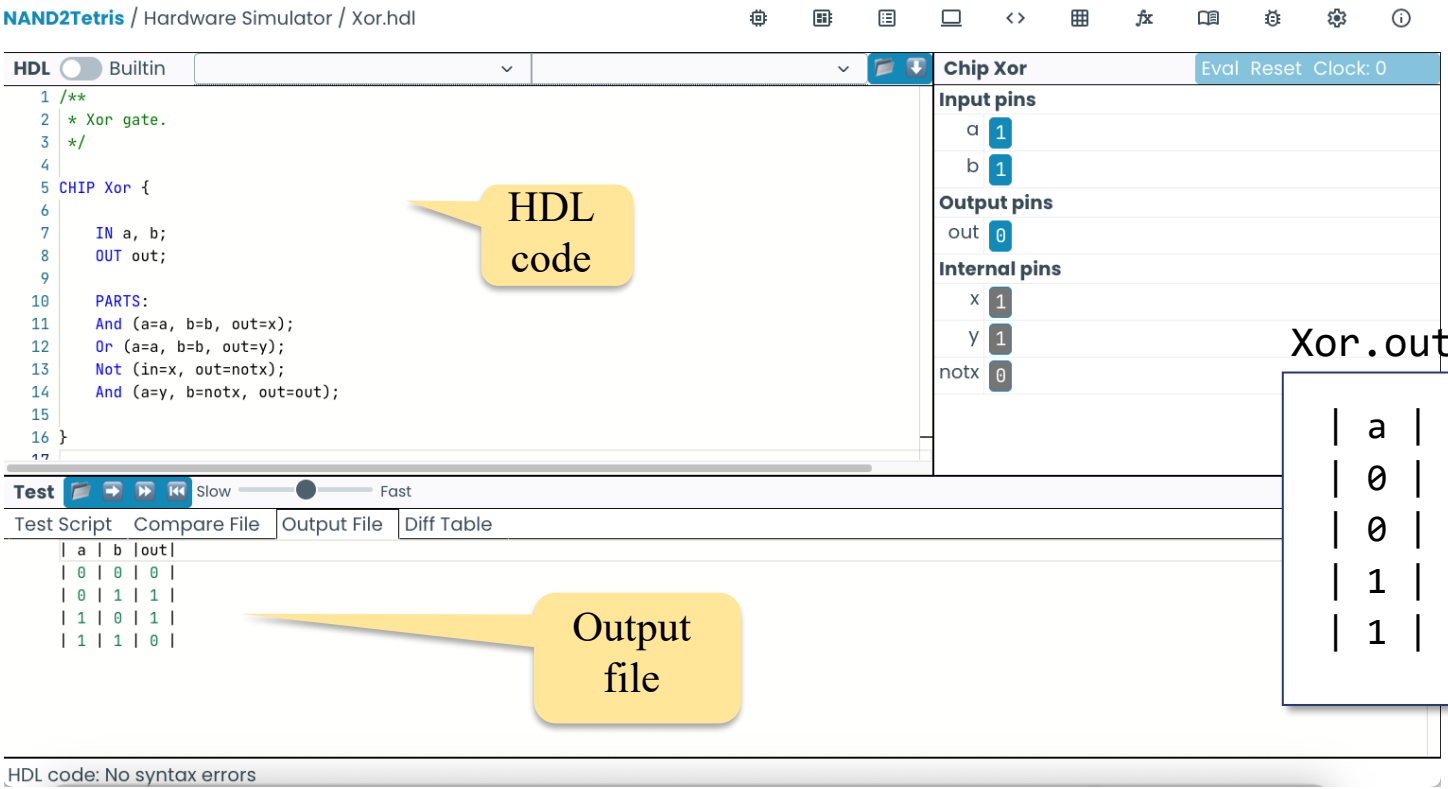
Script-based Testing

UNIVERSITY OF LEEDS



Script-based Testing

UNIVERSITY OF LEEDS



Xor.out

a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

Your turn

UNIVERSITY OF LEEDS

1. 在文本编辑器（文本编辑，VS代码）中为您的第一个芯片（first.tst）创建测试脚本

2. 然后将脚本加载到模拟器中。（您需要刷新浏览器，以便文件显示在列表中。）

3. 在模拟器中运行测试脚本，查看它是否产生预期的输出

first.out

```
| A | B | O |
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |
```

Script-based Testing

UNIVERSITY OF LEEDS

使用比较文件逻辑进行模拟

当执行每个输出命令时，将输出的行与比较文件中的相应行进行比较

如果两条线不相同，模拟器会抛出比较错误。

请注意，每列中的空格也必须匹配！

Xor.tst

```
load Xor.hdl,
output-file Xor.out,
compare-to Xor.cmp,
output-list a b out;
set a 0, set b 0, eval, output;
set a 0, set b 1, eval, output;
set a 1, set b 0, eval, output;
set a 1, set b 1, eval, output;
```

Xor.out

```
| a | b |out|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |
```

compare

Xor.cmp

```
| a | b |out|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |
```

Your turn

UNIVERSITY OF LEEDS

1) In a text editor create a compare file for your first chip.

It should look like the following.

first.cmp

```
| A | B | O |
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |
```

2) Update your first.tst file to include a compare file.

3) Run your script in the simulator and check that all comparisons are passed.

Your turn

UNIVERSITY OF LEEDS

- Now have a go at implementing the Half and Full Adders from Lecture 1.4.1.
  - 我们为文件HalfAdder.hdl和FullAdder.hdl以及所需的测试和比较文件提供了一个模板。
  - 提示：您可以使用您在其他HDL文件中创建的芯片。因此，一旦你创建了HalfAdder.hdl，你就可以将该芯片包含在FullAdder.hdl文件中，而无需复制hdl（因为HalfAdder 的芯片名称和文件名一致，所以能找到）

    Putting

    HalfAdder(a=a, b=b, sum=s1, carry=c1);

    in your FullAdder.hdl will include a Half Adder with pin names *a* and *b* and output pin names *s1* and *c1*.

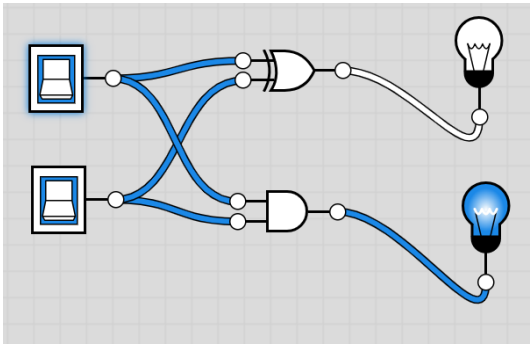
  - Below is a reminder of the circuits.

Your turn

UNIVERSITY OF LEEDS

- **Half Adder (HalfAdder.hdl)**
- Note the case of the pin names used for the inputs and outputs

a	b	sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

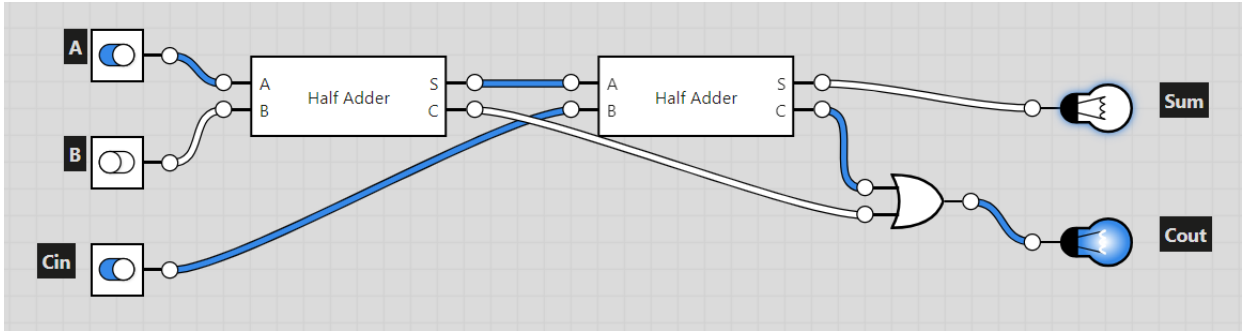


Your turn

UNIVERSITY OF LEEDS

- **Full Adder (FullAdder.hdl)**
- Note the names and case used for the input and output pins in the table

a	b	c	sum	carry
0	0	0	0	0
0	1	0	1	0
1	0	0	1	0
1	1	0	0	1
0	0	1	1	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	1



Your turn

UNIVERSITY OF LEEDS

Two binary numbers, a and b, each of two digits, are represented as a_1a_0 and b_1b_0 where a_1, a_0 and b_1, b_0 are Boolean variables which form the digits of the binary numbers.

Devise a combinatorial logic circuit, or circuits to provide outputs of X,Y, Z such that X=1 when a>b, Y=1 when a<b and Z=1 when a=b.

1. Start by constructing the 3 truth tables for the outputs X,Y and Z with inputs a1, a0, b1 and b0.
2. Derive(推导) the Boolean expressions for X,Y,Z from the truth table.
3. 用Kmap化简表达式
4. Implement the circuits in HDL. We have provided a template, test and cmp file for you, comparator.hdl, comparator.tst and comparator.cmp.

Summary

- Introduced Computer Aided Design (CAD) and Hardware Description Language (HDL)
- Introduced a hardware simulator
- Learnt how to write HDL files
- Learnt how to run simulations of simple logic circuits
- Implement the first two chips required for our HACK computer (HalfAdder and FullAdder)